

$x \sim p(x)$ · Generative Models

“What I cannot create, I do not understand.” — Richard Feynman

1 The goal: learning to sample

We have samples $x_1, x_2, \dots, x_N$ from some distribution $p_{\text{data}}(x)$ —images, molecules, protein structures. We want to learn a model that can generate *new* samples from this distribution.

The core idea: learn a **transformation** that maps samples from a simple distribution (like a Gaussian) to samples from the data distribution. If $z \sim \mathcal{N}(0, I)$ and we have a map T such that $T(z) \sim p_{\text{data}}$, then generating new samples is easy: sample z , apply T .

The question is: how do we learn T ?

2 Flows: transforming distributions

Change of variables. Let $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be an invertible, differentiable map. If z has density $p_0(z)$, then $x = T(z)$ has density

$$p_1(x) = p_0(T^{-1}(x)) \left| \det \frac{\partial T^{-1}}{\partial x} \right| \quad (2.1)$$

This is the **change of variables formula**. The Jacobian determinant accounts for how T stretches or compresses volume.

A **normalizing flow** parameterizes T with a neural network and trains it so that $p_1(x) \approx p_{\text{data}}(x)$. The appeal: we get exact likelihoods, not bounds. The challenge: computing the Jacobian determinant is expensive ($O(d^3)$ in general), so we need clever architectures.

Continuous normalizing flows. Instead of a single transformation, imagine a continuous family of transformations indexed by time $t \in [0, 1]$. Define a **velocity field** $v_t(x)$ and let x evolve according to the ODE:

$$\frac{dx}{dt} = v_t(x) \quad (2.2)$$

Starting from $x_0 = z \sim p_0$, integrating to $t = 1$ gives $x_1 = T(z)$.

The density evolves according to the **continuity equation**:

$$\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t v_t) = 0 \quad (2.3)$$

Equivalently, the log-density along a trajectory satisfies:

$$\frac{d \log p_t(x_t)}{dt} = -\nabla \cdot v_t(x_t) \quad (2.4)$$

We only need the *divergence* of v_t , not the full Jacobian—a significant computational savings.

The training problem. To train a continuous normalizing flow (CNF) by maximum likelihood, we need to:

1. Integrate the ODE from $t = 1$ (data) back to $t = 0$ (noise) to get $z = T^{-1}(x)$
2. Integrate the log-density change to get $\log p_1(x)$
3. Maximize $\sum_i \log p_1(x_i)$ over the training data

This requires **simulating the ODE at every training step**—expensive! Each gradient update needs a full forward and backward ODE solve. Can we do better?

3 Flow matching: simulation-free training

Flow matching [1] offers a way to train CNFs without simulation. The key insight: instead of maximum likelihood, directly regress the velocity field.

The flow matching objective. Suppose we knew the “true” velocity field $u_t(x)$ that transports p_0 to p_{data} . We could train a neural network $v_\theta(x, t)$ by simple regression:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x \sim p_t} [\|v_\theta(x, t) - u_t(x)\|^2] \quad (3.1)$$

No ODE solving needed—just sample (t, x) pairs and minimize squared error.

The problem. We don’t know $u_t(x)$! If we did, we wouldn’t need to learn anything. And computing $u_t(x)$ requires knowing p_t , which requires... solving the ODE we’re trying to avoid.

The solution: conditional flows. Here’s the trick. We don’t know the *marginal* velocity field $u_t(x)$, but we can construct *conditional* velocity fields $u_t(x|x_1)$ that are easy to compute.

Fix a data point x_1 . Define a simple path from noise to x_1 :

$$\psi_t(x_0|x_1) = (1 - t)x_0 + tx_1 \quad (3.2)$$

This is just linear interpolation: at $t = 0$ we’re at the noise x_0 , at $t = 1$ we’re at the data x_1 . The velocity along this path is:

$$u_t(x|x_1) = \frac{d\psi_t}{dt} = x_1 - x_0 \quad (3.3)$$

Remarkably simple: the conditional velocity is just the direction from noise to data.

From conditional to marginal. The marginal velocity field is the average of conditional velocities, weighted by how likely each x_1 is given the current position:

$$u_t(x) = \mathbb{E}_{x_1 \sim p(x_1|x_t=x)} [u_t(x|x_1)] \quad (3.4)$$

We can’t compute this expectation (it involves the intractable posterior $p(x_1|x_t)$). But we don’t need to! The **conditional flow matching** (CFM) objective:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_0 \sim p_0, x_1 \sim p_{\text{data}}} [\|v_\theta(\psi_t(x_0|x_1), t) - u_t(\psi_t(x_0|x_1)|x_1)\|^2] \quad (3.5)$$

samples from the *joint* $p_0(x_0)p_{\text{data}}(x_1)$ rather than the posterior.

Why does CFM work?

The CFM and FM losses have the same gradient with respect to θ . Intuitively: the conditional velocities $u_t(x|x_1)$ for different x_1 may point in different directions at the same x , but their average (weighted by likelihood) gives the correct marginal velocity. By regressing against all conditionals, we implicitly learn this average.

Formally, expanding the squared norm shows that the cross-terms are equal:

$$\mathbb{E}_{x_1}[\langle v_\theta, u_t(\cdot|x_1) \rangle] = \langle v_\theta, \mathbb{E}_{x_1}[u_t(\cdot|x_1)] \rangle = \langle v_\theta, u_t \rangle \quad (3.6)$$

The algorithm. Training is simple:

1. Sample $t \sim \text{Uniform}(0, 1)$
2. Sample noise $x_0 \sim \mathcal{N}(0, I)$ and data $x_1 \sim p_{\text{data}}$
3. Compute $x_t = (1 - t)x_0 + tx_1$
4. Train $v_\theta(x_t, t)$ to predict $x_1 - x_0$

Sampling is also simple: start from $x_0 \sim \mathcal{N}(0, I)$, integrate $dx/dt = v_\theta(x, t)$ from $t = 0$ to $t = 1$.

4 Gaussian paths and the variance schedule

The linear interpolation $x_t = (1 - t)x_0 + tx_1$ is one choice. More generally, we can use:

$$x_t = \alpha_t x_1 + \sigma_t x_0 \quad (4.1)$$

where α_t and σ_t are the **schedule**. Different schedules give different paths from noise to data.

Common schedules.

- **Flow matching (linear):** $\alpha_t = t$, $\sigma_t = 1 - t$. Simple, commonly used.
- **Variance preserving:** $\alpha_t^2 + \sigma_t^2 = 1$. Keeps $\|x_t\|^2$ roughly constant.
- **Variance exploding:** $\alpha_t = 1$, σ_t increases with t . Adds noise without scaling signal.

For Gaussian x_0 , the marginal $p_t(x_t|x_1)$ is also Gaussian:

$$p_t(x_t|x_1) = \mathcal{N}(x_t; \alpha_t x_1, \sigma_t^2 I) \quad (4.2)$$

This is the **Gaussian probability path**. The conditional velocity field for this path is:

$$u_t(x|x_1) = \frac{\dot{\alpha}_t}{\alpha_t} \left(x - \sigma_t^2 \frac{x - \alpha_t x_1}{\sigma_t^2} \right) + \dot{\sigma}_t \frac{x - \alpha_t x_1}{\sigma_t} \quad (4.3)$$

For the linear schedule, this simplifies to $u_t(x|x_1) = (x_1 - x)/(1 - t)$, or equivalently $x_1 - x_0$ when $x = x_t$.

5 The connection to diffusion models

Diffusion models and **Gaussian flow matching** are the same thing, viewed differently [2].

Diffusion forward process. Diffusion models define a forward process that adds noise:

$$x_t = \alpha_t x_1 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (5.1)$$

This is identical to the Gaussian probability path! The “noise” ϵ plays the role of x_0 .

Different parameterizations. Diffusion models typically train a network to predict:

- The noise ϵ (“ ϵ -prediction”)
- The clean data x_1 (“ x -prediction”)
- A combination $v = \alpha_t \epsilon - \sigma_t x_1$ (“ v -prediction”)

These are all linearly related: given $x_t = \alpha_t x_1 + \sigma_t \epsilon$, we can convert between them:

$$\epsilon = \frac{x_t - \alpha_t x_1}{\sigma_t}, \quad x_1 = \frac{x_t - \sigma_t \epsilon}{\alpha_t} \quad (5.2)$$

Flow matching predicts the velocity $u_t = x_1 - x_0 = x_1 - \epsilon$ (for linear schedule). This is yet another linear combination.

Equivalence of samplers. The DDIM sampler [3] for diffusion models:

$$x_{t-\Delta t} = \alpha_{t-\Delta t} \hat{x}_1 + \sigma_{t-\Delta t} \hat{\epsilon} \quad (5.3)$$

where $\hat{x}_1$ is the network’s prediction of clean data from x_t . This is exactly the Euler method for the flow matching ODE, just reparameterized.

The equivalence

For Gaussian source distributions:

Diffusion models = Flow matching

They differ only in:

- What the network outputs (ϵ , x_1 , v , or velocity)
- The noise schedule (α_t , σ_t)
- Sampling (deterministic ODE vs. stochastic SDE)

These are implementation choices, not fundamental differences.

6 Score functions and the SDE perspective

The score function. The **score** is the gradient of the log-density:

$$s_t(x) = \nabla_x \log p_t(x) \quad (6.1)$$

It points toward regions of higher probability. For the Gaussian path:

$$\nabla_x \log p_t(x|x_1) = -\frac{x - \alpha_t x_1}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t} \quad (6.2)$$

So predicting the noise ϵ is equivalent to predicting the score (up to scaling).

The SDE formulation. The forward process can be written as a stochastic differential equation:

$$dx = f_t x dt + g_t dW \quad (6.3)$$

where dW is Brownian motion. The reverse process is also an SDE:

$$dx = [f_t x - g_t^2 \nabla_x \log p_t(x)] dt + g_t d\bar{W} \quad (6.4)$$

where $d\bar{W}$ is reverse-time Brownian motion.

ODE vs. SDE sampling. Setting the noise to zero in the reverse SDE gives the **probability flow ODE**:

$$dx = [f_t x - \frac{1}{2} g_t^2 \nabla_x \log p_t(x)] dt \quad (6.5)$$

This ODE has the same marginal distributions p_t as the SDE, but follows deterministic trajectories. The probability flow ODE *is* a continuous normalizing flow—the connection is exact.

Stochastic vs. deterministic sampling. With the ODE (deterministic), each noise sample maps to exactly one output. With the SDE (stochastic), we add noise during sampling, exploring more of the distribution. The SDE can improve sample quality by correcting errors, at the cost of slower sampling.

7 Why flow matching works well

Simulation-free training. Unlike maximum likelihood for CNFs, flow matching never solves an ODE during training. Each training step is a simple regression.

The “straightness” intuition. If the model perfectly predicts each data point, the conditional paths $x_t = (1 - t)x_0 + tx_1$ are straight lines. Straight ODEs have zero integration error.

But this intuition is misleading. The *marginal* paths (what the trained model actually follows) are generally curved, because the model predicts an average over possible endpoints, not a single point. The optimal schedule depends on the data distribution—linear isn’t always best.

Crossing paths and variance. A subtlety: conditional paths for different (x_0, x_1) pairs can cross. At a crossing point, the model sees conflicting supervision (different velocities for the same x_t). This increases gradient variance and can slow training.

Optimal transport couplings address this by pairing x_0 and x_1 to minimize crossings. Instead of independent sampling, use:

$$(x_0, x_1) \sim \pi^*(x_0, x_1) = \arg \min_{\pi} \mathbb{E}_{\pi}[\|x_0 - x_1\|^2] \quad (7.1)$$

This is the optimal transport plan. In practice, we approximate it over mini-batches [4].

8 Latent variable models: a different approach

Flow matching transforms noise to data directly. **Variational autoencoders** (VAEs) take a different route: learn a low-dimensional latent space.

The idea. Instead of transforming $\mathbb{R}^d \rightarrow \mathbb{R}^d$, compress data into a latent $z \in \mathbb{R}^k$ with $k \ll d$, then reconstruct. The encoder $q_{\phi}(z|x)$ maps data to latents; the decoder $p_{\theta}(x|z)$ maps latents to data.

The ELBO. We can't compute $\log p_{\theta}(x) = \log \int p_{\theta}(x|z)p(z)dz$ directly. Instead, we optimize a lower bound:

$$\log p_{\theta}(x) \geq \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - \text{KL}(q_{\phi}(z|x)||p(z)) \quad (8.1)$$

The first term encourages reconstruction; the second regularizes the latent space toward the prior $p(z) = \mathcal{N}(0, I)$.

Connection to flow matching. Diffusion models can be viewed as hierarchical VAEs with many latent layers [5]. Each noise level is a latent variable; the forward process is a fixed encoder; the reverse process is the learned decoder. The ELBO for diffusion decomposes into terms for each noise level, which (after simplification) gives the denoising objective.

9 From VAEs to diffusion: making the problem harder

VAEs learn a compressed representation. Diffusion models do something counterintuitive: they make the latent space *the same size as the data*.

Why does this help? The challenge in generative modeling is the gap between simple distributions (Gaussians) and complex ones (images). VAEs try to bridge this gap with a single encoder-decoder pair. Diffusion breaks it into many small steps.

Each denoising step—predicting slightly cleaner data from noisy data—is an easy problem. The full generative process chains thousands of easy steps into one hard transformation. This is the “making hard problems harder” trick: by expanding the problem (adding many intermediate steps), we make each piece tractable.

The key insight. Diffusion models succeed because:

1. Each step is simple (predict signal from signal + noise)
2. Steps share parameters (one network for all noise levels)
3. Training is amortized (random timestep per example)
4. The forward process allows exact marginalization (jump to any t)

10 Practical considerations

Network architecture. The velocity/score network takes (x_t, t) as input. For images, U-Nets are standard; for other domains, transformers or graph networks. The time t is typically encoded via sinusoidal embeddings and injected via adaptive normalization.

Training objective weighting. The CFM loss weights all timesteps equally. In practice, different weightings can improve sample quality:

$$\mathcal{L} = \mathbb{E}_t[w(t)\|v_\theta(x_t, t) - u_t\|^2] \quad (10.1)$$

Common choices: $w(t) = 1$ (uniform), $w(t) \propto \text{SNR}(t)$ (signal-to-noise ratio), or learned weightings.

Sampling speed. Naive Euler integration needs many steps (hundreds to thousands). Faster options:

- Higher-order ODE solvers (Heun, RK45)
- Distillation: train a student to match the teacher in fewer steps
- Consistency models: train to map any x_t directly to x_1

Classifier-free guidance. For conditional generation (e.g., text-to-image), train with conditioning c sometimes dropped. At sampling, extrapolate:

$$\tilde{v}_\theta(x_t, t, c) = v_\theta(x_t, t, \emptyset) + w \cdot (v_\theta(x_t, t, c) - v_\theta(x_t, t, \emptyset)) \quad (10.2)$$

with guidance weight $w > 1$. This sharpens the distribution toward the condition.

11 Summary

The landscape of generative models:

Normalizing flows. Invertible transformations with exact likelihoods. Training by maximum likelihood requires Jacobian computation; continuous flows need ODE simulation.

Flow matching. Simulation-free training of continuous flows. Regress the velocity field against simple conditional targets. The key trick: conditional flow matching gives the same gradients as marginal flow matching.

Diffusion models. Equivalent to Gaussian flow matching. Different parameterizations (ϵ , x , v , velocity) and schedules, but the same underlying algorithm. Can sample with deterministic ODE or stochastic SDE.

VAEs. Latent variable models with encoder and decoder. Train via the ELBO. Diffusion can be viewed as a hierarchical VAE with fixed encoder.

The unifying view. All these methods learn to transform simple distributions into complex ones. They differ in:

- The path (direct transformation vs. many small steps)
- The training objective (likelihood vs. regression)
- The computational tradeoffs (Jacobians vs. simulation vs. neither)

Flow matching has emerged as a simple, flexible framework that combines the best of these approaches: exact continuous flows, simulation-free training, and connection to the rich toolkit of diffusion models.

References

- [1] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [2] Ruiqi Gao, Emiel Hoogeboom, Jonathan Heek, Valentin De Bortoli, Kevin P. Murphy, and Tim Salimans. Diffusion meets flow matching: Two sides of the same coin, 2024. URL <https://diffusionflow.github.io/>.
- [3] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.
- [4] Alexander Tong, Nikolay Malkin, Guillaume Huguette, Yanlei Zhang, Jarrod Rector-Brooks, Kilian Fatras, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.
- [5] Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. *Advances in Neural Information Processing Systems*, 34:21696–21707, 2021.